

BCS 2410: Embedded Programming (2025-2026)

1. Course description

This course (4 ECTS) introduces the fundamental concepts and practical skills required to develop software for embedded systems. Embedded systems are specialized computing devices designed to perform specific tasks within larger systems and are commonly found in industries such as automotive, healthcare, consumer electronics, robotics, digital agriculture and the Internet of Things (IoT). As part of this course, you will learn about microcontrollers, memory management, peripheral interfacing and power management techniques, using low-level programming languages such as C and assembly. The course also provides an introductory overview of FPGA-based systems and hardware-accelerated computing. You will be introduced to Vitis AI as a toolchain for deploying and accelerating simple machine learning workloads on FPGA-based embedded platforms.

Topics include:

- Computer and microcontroller architecture: Overview of hardware, including CPUs, memory, timers, and hardware/software interrupts
- Programming for embedded systems: Using C and assembly to write basic, low-level code for microcontrollers
- Interfacing with peripherals: Communication protocols like GPIO control
- Introduction to FPGA-based embedded systems: Fundamental concepts of reconfigurable logic, FPGA architectures and their role in embedded computing
- Introduction to AI Acceleration on FPGA: Overview of Vitis AI and FPGA-based acceleration

By the end of the course, you will be able to design and implement basic software for embedded systems. This course synergizes well with the Ubiquitous Computing & Internet of Things course as well as with Project 2-2 on Cybersecurity & IoT.

2. Teacher information

Lecturers and tutors

- | | | |
|------------------------|--|--------|
| • Charis Kouzinopoulos | charis.kouzinopoulos@maastrichtuniversity.nl | C4.011 |
| • Angela Cratere | angela.cratere@maastrichtuniversity.nl | C4.029 |

Tutors/Teaching Assistants

- Lorenzo Bianchi (l.bianchi@student.maastrichtuniversity.nl)
- Derrick Timmermans (dxy.timmermans@student.maastrichtuniversity.nl)
- Luuk Janssen (lj.janssen@student.maastrichtuniversity.nl)
- Daniel Marcon (d.marcon@student.maastrichtuniversity.nl)

How to find us?

We will use Canvas for important announcements. Other than that, we are mostly available online, via our email addresses (see above). We aim to always respond to your emails quickly, typically after 10-30 minutes to 2-3 working days. However, due to the increasing number of students, this time could be longer in some periods. If you ever wait longer than 1 week without hearing from us, please mail us a reminder. You will probably receive a more helpful answer from us if your email contains at least one clearly-phrased question or request. A reference to the course in the subject of the email (e.g. EPr - Doubt about Module 3) would be helpful as well.

If you want to discuss something in person, you can ask us after class. You can also -usually- find us in our DACS offices (PHS1 building) but due to our busy schedules, it's always good to check via email (or ask for an appointment).

3. Textbook and Resources

Textbook(s):

- David A. Patterson, John L. Hennessy - Computer Organization and Design ARM Edition: The Hardware Software Interface
- Y. Zhu - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C (Fourth Edition)
- D. Harris, S. Harris - Digital Design and Computer Architecture

Slides and exercises:

- Available online via Canvas

Software:

- STM32CUBE (Check Assignment 0 for instructions)
- Vitis AI 3.5

Hardware:

- [B-U585I-IOT02A Discovery kit](#)

We will also use parts of datasheets by ARM and ST to determine various characteristics of our embedded system. Datasheets play a crucial role in embedded programming since they provide technical specifications about microcontrollers, sensors, memory devices, communication modules and peripherals so it is important to understand how to use them properly!

- STM32U585AI datasheet <https://www.st.com/resource/en/datasheet/stm32u585ai.pdf>
- PM0264 - STM32 Cortex-M33 MCUs programming manual
https://www.st.com/resource/en/programming_manual/pm0264-stm32-cortexm33-mcus-programming-manual-stmicroelectronics.pdf
- Arm Cortex-M33 Processor Technical Reference Manual
<https://developer.arm.com/documentation/100230/latest/>
- Armv8-M Architecture Reference Manual
<https://developer.arm.com/documentation/ddi0553/latest/>
- Vitis AI User Guide (UG1414)
<https://docs.amd.com/r/en-US/ug1414-vitis-ai>
- DPUCZDX8G for Zynq UltraScale+ MPSoCs Product Guide (PG338)
https://docs.amd.com/r/en-US/pg338-dpu?tocId=3xsG16y_QFTWvAJKHbisEw

4. Course format and composition

In this course, we are going to be using a format which includes lectures, both classic and live-coding, individual study at home (including additional video material, book chapters, quizzes, homework, etc.), on-site (practical) tutorials and dedicated sessions (e.g recap and exam preparation lectures). Some of the sessions (Q&A sessions) are support sessions. They do not cover new material but focus on revising previous concepts and thus are dedicated to students facing problems with the concepts covered in the course. Below we explain the different components.

Live Coding Lectures: During live coding lectures you will be instructed in the basics of new concepts via simple illustrative examples. You are expected to engage with the lecturer and your fellow classmates by asking questions.

Individual study (Video material, quizzes, homework): Each week, we will provide some additional material (as videos or other formats) which we expect you to cover individually at home. Our expectation is that you cover part of this material before the relevant live coding lecture, so that you are prepared and able to identify how the concepts you studied are applied in practice. Each live coding lecture is followed by small quizzes, where you test your knowledge, plus some homework exercises to practice. Ideally, you should have finished homework before coming to the next tutorial, but due to the tight schedule, it will not always be feasible. Try, at least, to have it prepared. Homework is not mandatory and is not to be delivered (or graded) but will help you build the foundation needed to attempt the practical sessions.

Tutorials: These sessions are intended as hands-on sessions where you, together with other colleagues and guided by a TA will work on a programming task on the Discovery board.

Q&A sessions: The two Q&A sessions are intended to solve conceptual and code-related doubts.

Office hours: The office hours are intended to solve conceptual doubts. For coding-related issues, you must use other channels, namely the Ask a TA sessions. Each office hours session is dedicated to a specific module and doubts related to it will be prioritized. Therefore, clarifying doubts related to other (previous) modules is not assured.

5. Study workload

This course is worth **4 ECTS** points, which equates to **112 hours of study**. Based on this, we built the weekly plan below. As indicated there, we expect 8-12 hours of self-study (pre+post lecture material, homework, etc.) per week. The estimated workload can be summarized as follows:

- 20 contact hours during Lectures and Live coding sessions
- 10 contact hours for tutorials
- 6 preparation hours (before each lecture/live coding session)
- 6 review hours (after each lecture/live coding session)
- 10+ hours for the exam preparation

Additionally, you have the chance to attend (contact hours):

- 10 office hours
- 2 Q&A sessions with TAs for coding-related doubts

Rough weekly plan

Week	Day	Hours	Content	Type	Who
1	Monday 26/01	2	Introduction, computer architecture	Lecture / Intro	Angela
	At your own pace	2		Review material	You!
	Wednesday 28/01	2		Lecture	Angela
	At your own pace	3		Review material / quizzes	You!
	Thursday 29/01	2		Lab	TA
2	Monday 02/02	2	Computer architecture, data representation and bitwise operations	Lecture / Intro	Charis
	At your own pace	2		Review material	You!
	Wednesday 04/02	2		Lecture	Charis
	At your own pace	3		Review material / quizzes	You!
	Thursday 05/02	2		Lab	TA
3	Monday 09/02	2	Pointers and the ARM ISA	Lecture / Intro	Charis
	At your own pace	2		Review material	You!
	Wednesday 11/02	2		Lecture	Charis
	At your own pace	3		Review material / quizzes	You!
	Thursday 12/02	2		Lab	TA
4	Monday 23/02	2	The ARM ISA	Lecture / Intro	Charis
	At your own pace	2		Review material	You!
	Wednesday 25/02	2		Lecture	Charis
	At your own pace	3		Review material / quizzes	You!
	Thursday 26/02	2		Lab	TA
5	Monday 02/03	2	The STM32U5 architecture, GPIOs, intro to FPGAs	Lecture / Intro	Charis
	At your own pace	2		Review material	You!
	Thursday 05/03	2		Lecture	Angela
	At your own pace	3		Review material / quizzes	You!
	Friday 06/03	2		Lab	TA
6	Monday 09/03	2	Intro to FPGAs, content revision	Lecture / Intro	Angela
	At your own pace	2		Review material	You!
	Wednesday 11/03	2		Lecture	Charis
	At your own pace	3		Review material / quizzes	You!

6. Intended Learning Outcomes and Learning Goals

The Intended Learning Outcomes (ILOs) of the EPr course are the following:

Intended learning outcomes (ILOs)	Teaching & learning activities (TLAs)
Knowledge and understanding: Students will be able to explain the methodological and theoretical principles of embedded programming	Classic lectures
Applying knowledge and understanding: Students will be able to implement basic, embedded programs on 32-bit systems using assembly and C	Programming exercises
Making judgement: Students will be able to judge the quality and correctness of basic embedded programs	Classic lectures Pair programming exercises
Communication: Students can discuss (oral and written) the correct use of standard programming interfaces and algorithmic basics	Classic lectures Programming exercises
Learning skills: Students learn to recognize their own lack of knowledge and understanding and take appropriate action such as consulting additional material or other sources of help	Programming exercises

These ILOs can be split into more concrete Learning Goals (LGs). The summary of these LGs, grouped by course module, is shown below:

- Module 1: Introduction to computer architecture
 - You know the definition of embedded systems and the basic details of the ARM processor family
 - You understand the basic characteristics of the B-U585I-IOT02A discovery board
 - Processor
 - Memory
 - Peripherals
 - You understand the language of computers
 - You understand the importance and functionality of transistors
 - You understand how computers are organized and what is the power wall
 - You understand the Von Neumann and Harvard architectures
 - You understand the role of the ALU
 - You know the main memory types and the roles and principles of the cache:
 - Mapping functions
 - Replacement algorithms
- Module 2: Computer architecture, data representation and bitwise operations
 - You understand the functions of registers
 - You know how to create simple datapaths
 - You understand the concept of instruction cycles and pipelining
 - You understand the concept of number systems and how to convert numbers between different bases
 - You understand the basic C datatypes and their sizes
 - You know the role of the Application Program Status Register
 - You understand addition/subtraction for unsigned and signed integers and for binary numbers:
 - Two's complement
 - You know the basic bitwise operations:
 - AND
 - OR
 - XOR

- Shift
- You understand how to use bitwise operations to:
 - Set, toggle, test and clear specific bits
- Module 3: Pointers and the ARM ISA
 - You understand the difference between little endian vs big endian addressing
 - You know the function and size of pointers:
 - Reference operations
 - Dereference operations
 - You understand how to perform pointer arithmetic and the relation between pointers and arrays
 - You understand the concept of call by reference
 - You know the difference between RISC and CISC
 - You understand the syntax and usage of assembly instructions:
 - Arithmetic and logic: Add, subtract, bitwise operations, shift, rotate
 - Data movement: Load, Store, Move
- Module 4: The ARM ISA
 - You understand the way procedures are defined and used in ARM
 - You understand the concept of stack and its usage
 - You understand the syntax and usage of assembly instructions:
 - Compare and branch: Compare, Test, If-then, Branch, compare and branch on zero
- Module 5: The STM32U5 architecture and GPIOs
 - You know how the memory map of STM32U5 is partitioned
 - You understand how the processor core interconnects with the memory and the different peripherals
 - You understand the basic structure of the device GPIOs:
 - Input GPIOs
 - Output GPIOs
 - You know the different clocks and options related to the B-U585I-IOT02A discovery board for:
 - LEDs
 - push buttons
- Module 6: Introduction to FPGA architecture, the Deep Learning Processing Unit (DPU) technology and the Vitis AI development environment
 - You know the basic architecture of an FPGA
 - You understand the concept of hardware acceleration
 - You understand the role of DPUs as dedicated hardware accelerators for neural network inference on FPGA-based platforms
 - You know the high-level architecture of a DPU
 - You understand the Vitis AI development flow

7. Assessment

We expect all students to complete the quizzes and participate in class. Students are evaluated through a final exam taken using ANS which is graded out of 10. The course assignments or any other proposed exercises such as quizzes do not count for the final grade.

For any multiple choice questions, a correction for guessing will be applied:

$$\text{Adjusted score} = P \times \left(C \frac{W}{k-1} \right)$$

Where P the points per correct answer, C the number of correct answers, W the number of wrong answers and k the number of options per question. For example, for 8 multiple choice questions, with 0.5 points per question and 4 options per question, the adjusted score with 6 correct answers and 2 wrong questions would be:

$$0.5 \times \left(6 \frac{2}{3} \right) = 2$$

Note: Unanswered questions are simply ignored (they count as 0 correct and 0 wrong).

The structure of the exam can be estimated as follows:

- Theoretical questions + small coding exercises: ~40%
- Practical exercises (usually, 2-3 exercises): ~60%

The final grade is computed as follows:

$$\text{final grade (out of 10)} = \text{exam grade}$$

Resits

In the case of no passing grade during the normal period, you can take a resit exam (same format as the normal period exam). In this case, your final grade will be computed as follows:

$$\text{resit final grade (out of 10)} = \text{resit exam}$$

8. Class Policies (diversity/inclusivity, cheating)

Behavior in Class (online and offline)

- We do not tolerate any form of discrimination or violence in classes. If we observe or hear about a person in class attacking, harassing or ridiculing someone else because of their outer appearance, language, age, origin, gender, sexual orientation, religion, nationality, abilities, or because of physical or mental impairments or disabilities, we will take action to stop this kind of behavior.
- We expect you to respect other participants in class, and to avoid any behavior that might impact other students' ability to do well in the class. This includes talking loudly or distracting people during lectures. You should feel free to turn your attention elsewhere if you want, but we expect you to never do so at the expense of other students' ability to focus on the class.
- Our own behavior in class should follow the rules given above, just as much as yours. In case we say or do something that contradicts these guidelines, please point out the mistake and be prepared to discuss the issue with us, either in class or in private.

Cheating/Honor Code

- No code or solutions are to be distributed to other students either electronically (i.e. e-mail) or on paper or posted online where they can easily be discovered (i.e. Facebook, webpages, forums, discussion groups, etc.)
- Be sure that you read the terms of the license for the code that you are reusing. Most code that is found openly on the internet has specific licenses that you should not violate.
- When there is doubt regarding the honorability of an action, you need to ask!

9. Tips & Tricks for the course

- If you are having trouble with the contents of one lecture/tutorial, you can catch up on that lecture/tutorial at home. You can talk to other class participants, go through the lecture materials, or use the recommended or other online resources, or books to read up on specific topics. If, **after checking all the material proposed**, you need additional materials, feel free to talk to us. We will be happy to provide resources as needed.
- If you are unhappy throughout several sessions (lectures/tutorials) of the course and feel like you are not progressing, do not hesitate to talk to us. You can contact us in person or via email (see above how you can contact us). Together, we can figure out what the problem is and find solutions that work for you.
- Course lecturers enjoy teaching and are looking forward to teaching you the basics of embedded programming. Any feedback for us is valuable so feel free to contact us.